香港科技大學(廣州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

AIAA 2205: Introduction to AI

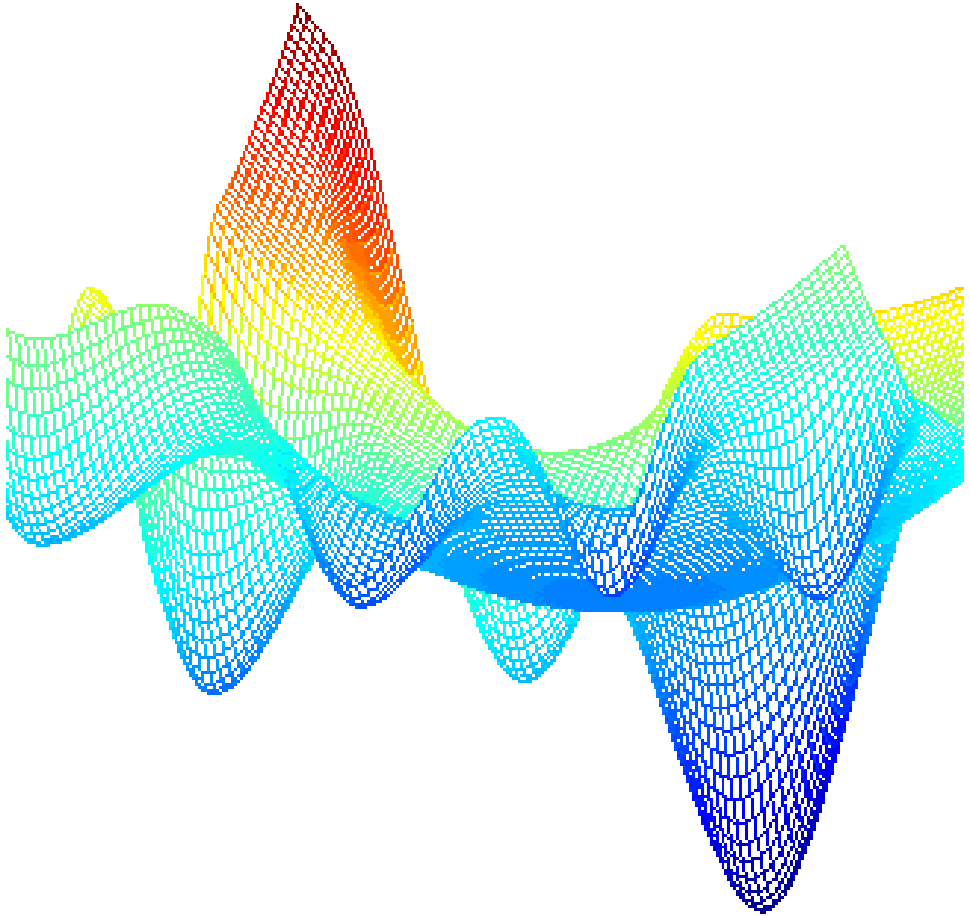
Optimization

Li LIU

avrillliu@hkust-gz.edu.cn

The Hong Kong University of Science and Technology (Guangzhou)

Treasure Hunting in the Mountains
山里寻宝



- Global minimum
- Local minimum
- Saddle point

Is the prediction correct or not?

Prediction equals ground truth or not

$I \in R^{32\times32\times3}$

$\boldsymbol{x} \in R^{3072\times1}$

vectorization

$f(\boldsymbol{x}, \boldsymbol{W}, \boldsymbol{b})$

$\boldsymbol{W}, \boldsymbol{b}$

parameters

10 numbers giving class scores

Prediction: largest score

What W (weight) are we looking for?

- The W that gives us the lowest L (loss function value).

Is the model a good model?

the "loss function" large or small? It summarizes the performance of a batch of inputs and outputs

$I \in R^{32\times32\times3}$

$\boldsymbol{x} \in R^{3072\times1}$
vectorization

$f(\boldsymbol{x}, \boldsymbol{W}, \boldsymbol{b})$

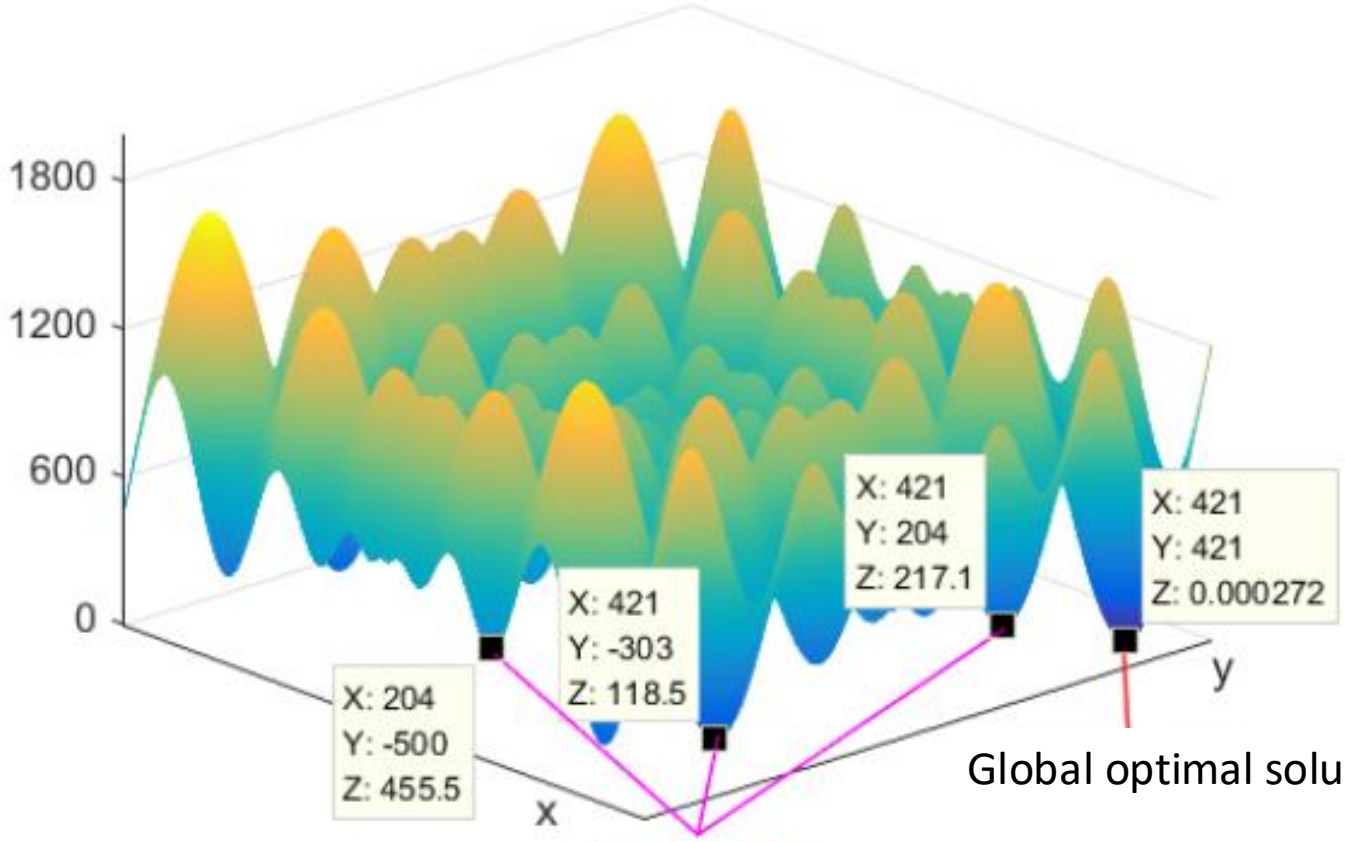
$\boldsymbol{W}, \boldsymbol{b}$
parameters

10 numbers giving class scores

Prediction: largest score

Training ≈ Minimizing loss function

Loss function is affected by a lot of w values (variable of the function), and hyperparameters



Global optimal solution

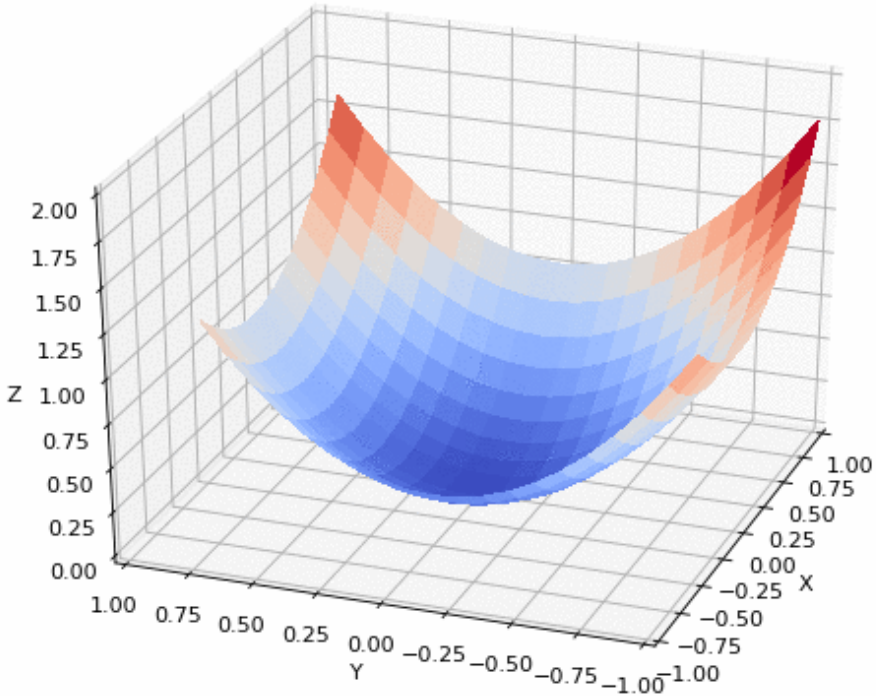
Local optimal solution

Optimization

Closed-form solution?

Gradient descent

Gradient descent



Idea: follow the direction that leads to steepest loss reduction.

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \lambda \frac{\partial L}{\partial \boldsymbol{W}}$$

Step size

Gradient

Gradient descent

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \lambda \frac{\partial L}{\partial \boldsymbol{W}} = \boldsymbol{W} - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^{N} L_i}{\partial \boldsymbol{W}} = \boldsymbol{W} - \lambda \frac{1}{N} \sum_{i=1}^{N} \frac{\partial L_i}{\partial \boldsymbol{W}}$$

$$L_i = -\sum_{j=1}^{K} y_j \log(p_j)$$

- K is the #class, y is one-hot vector
- There are many different kinds of losses

Converting prediction score to probability

Probabilities must be >= 0

Probabilities must sum to 1

cat	**3.2**	exp	**24.5**	normalize	**0.13**
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

$s = f(x_i, W)$ $\quad e^s$ $\quad \frac{e^{s_k}}{\sum_j e^{s_j}}$

Softmax function

Loss function

Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

Probabilities must be >= 0

Probabilities must sum to 1

cat	**3.2**	exp	**24.5**	normalize	**0.13**	compare	**1**
car	5.1		164.0		0.87		**0**
frog	-1.7		0.18		0.00		0

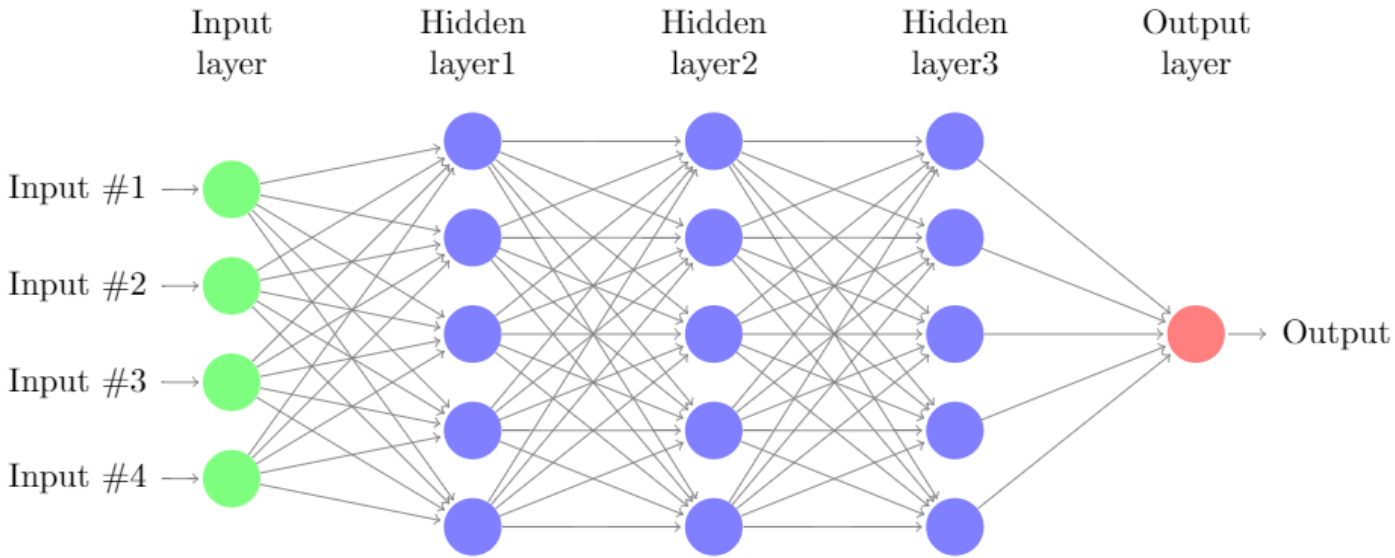
$s = f(x_i, W)$ $\quad e^s$ $\quad \frac{e^{s_k}}{\sum_j e^{s_j}}$

How do we get the value of W and b?

- Define a loss function
  - Softmax classifier

- Optimize W and b to minizmize the loss function

Backpropagation: computing the gradient


Input layer
Hidden layer1
Hidden layer2
Hidden layer3
Output layer
Input #1
Input #2
Input #3
Input #4
Output

Computation Graph

Assume $\mathrm{L} = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

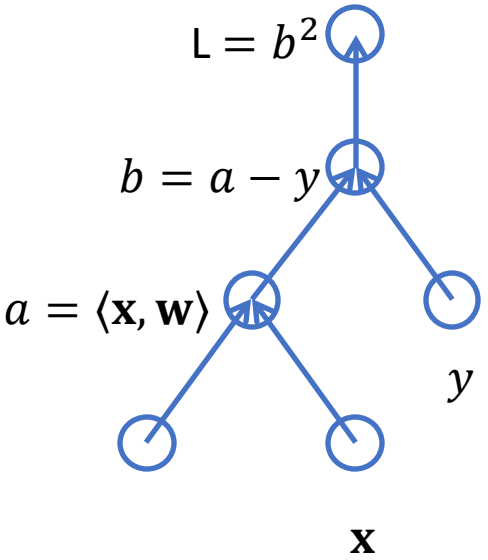
Forward

- Decompose into primitive operations
- Build a directed acyclic graph to present the computation

Backword

- Get the gradient by chain rule

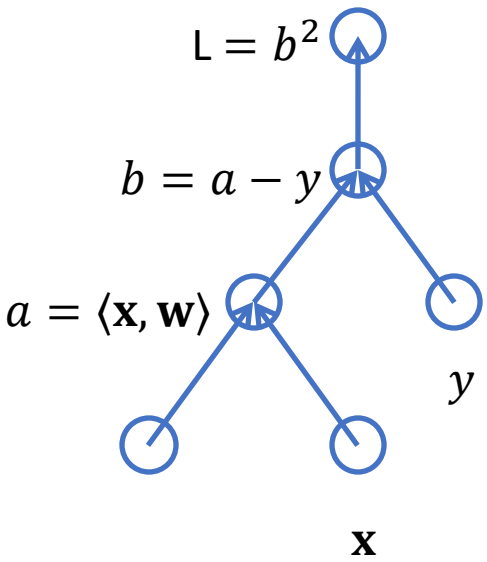
$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \cdots \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x}$$



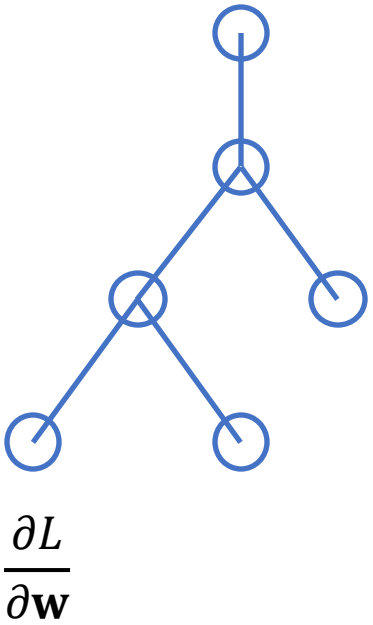
Reverse Accumulation

Assume $\mathrm{L} = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

Forward

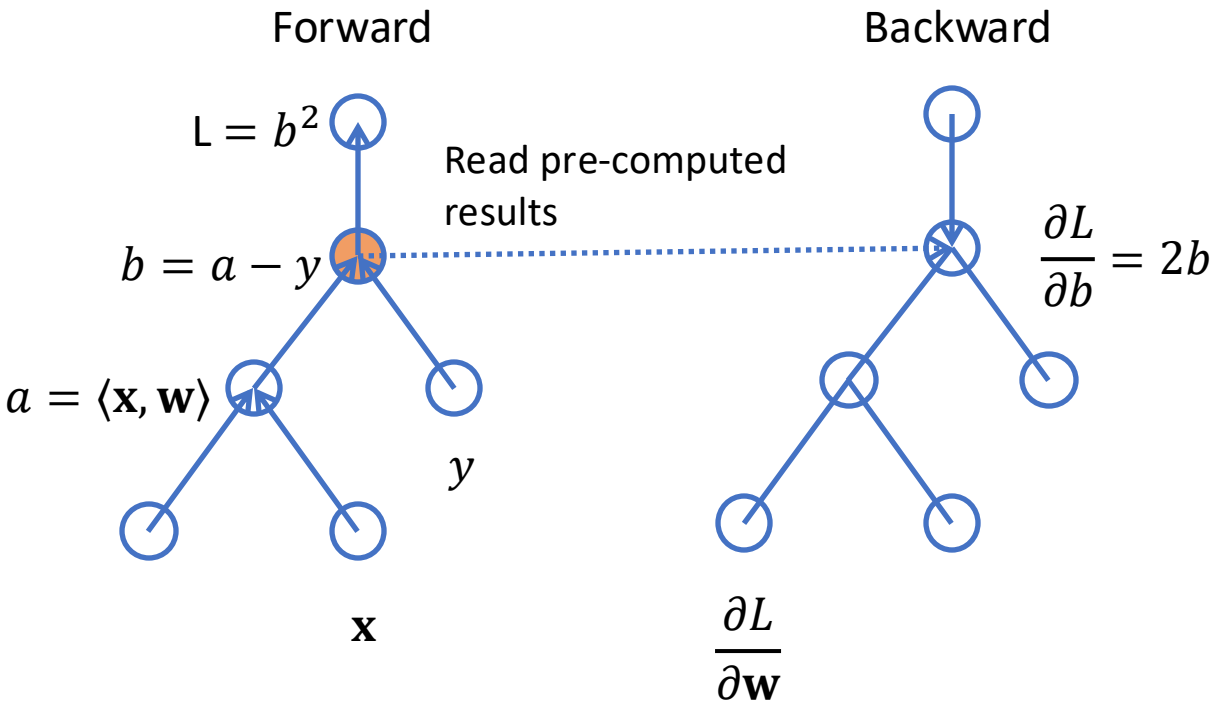


Backward



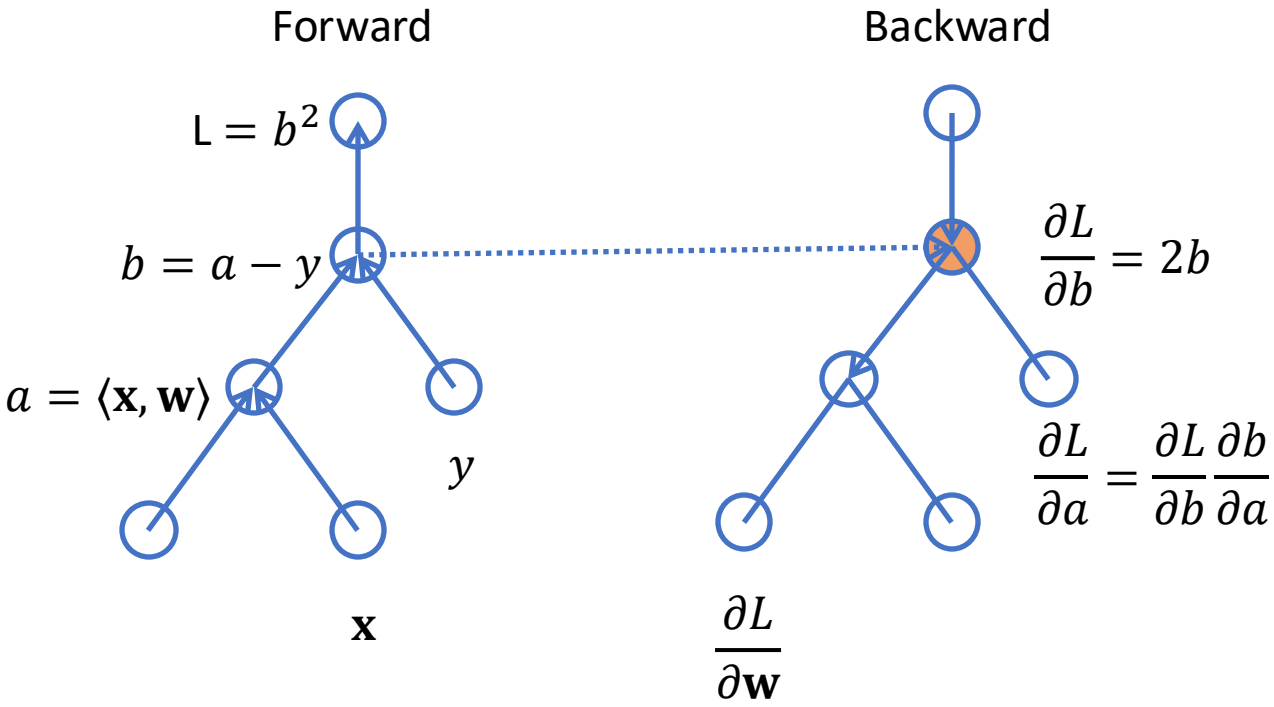
Reverse Accumulation

Assume $\mathrm{L} = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



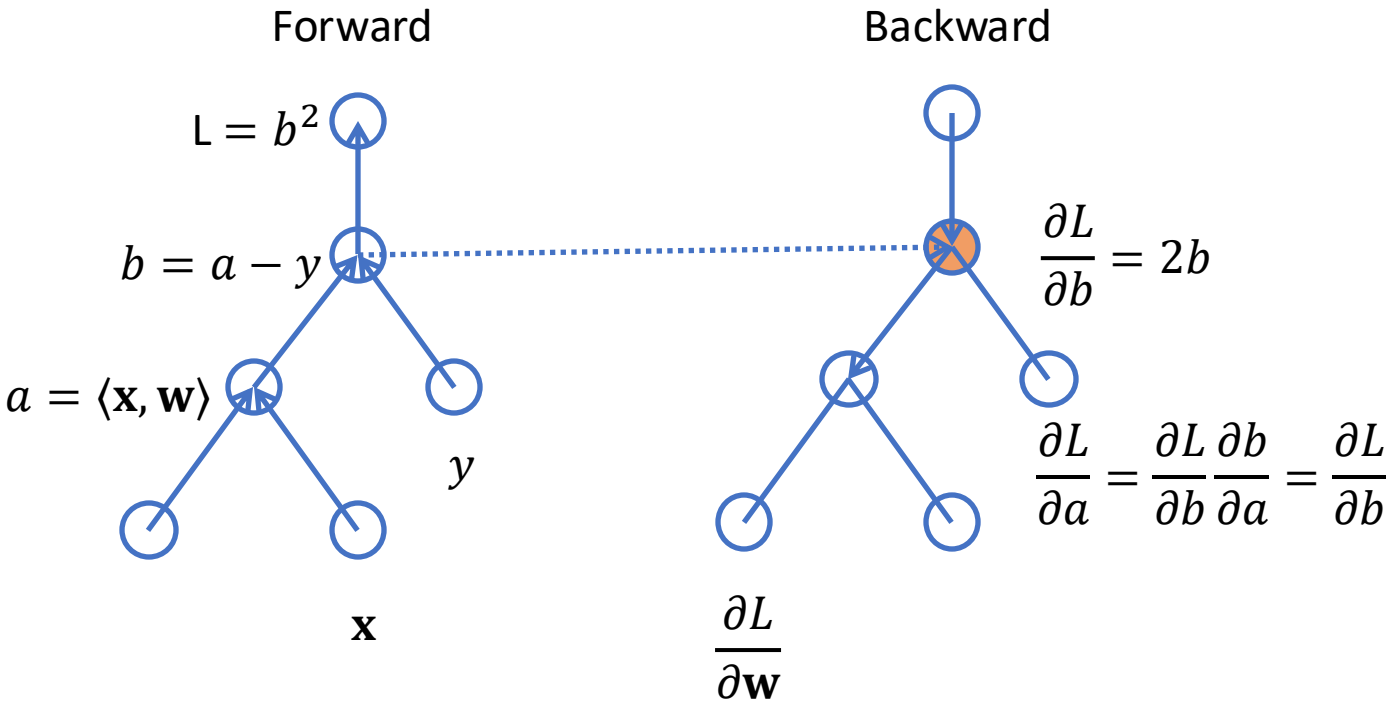
Reverse Accumulation

Assume $\mathrm{L} = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



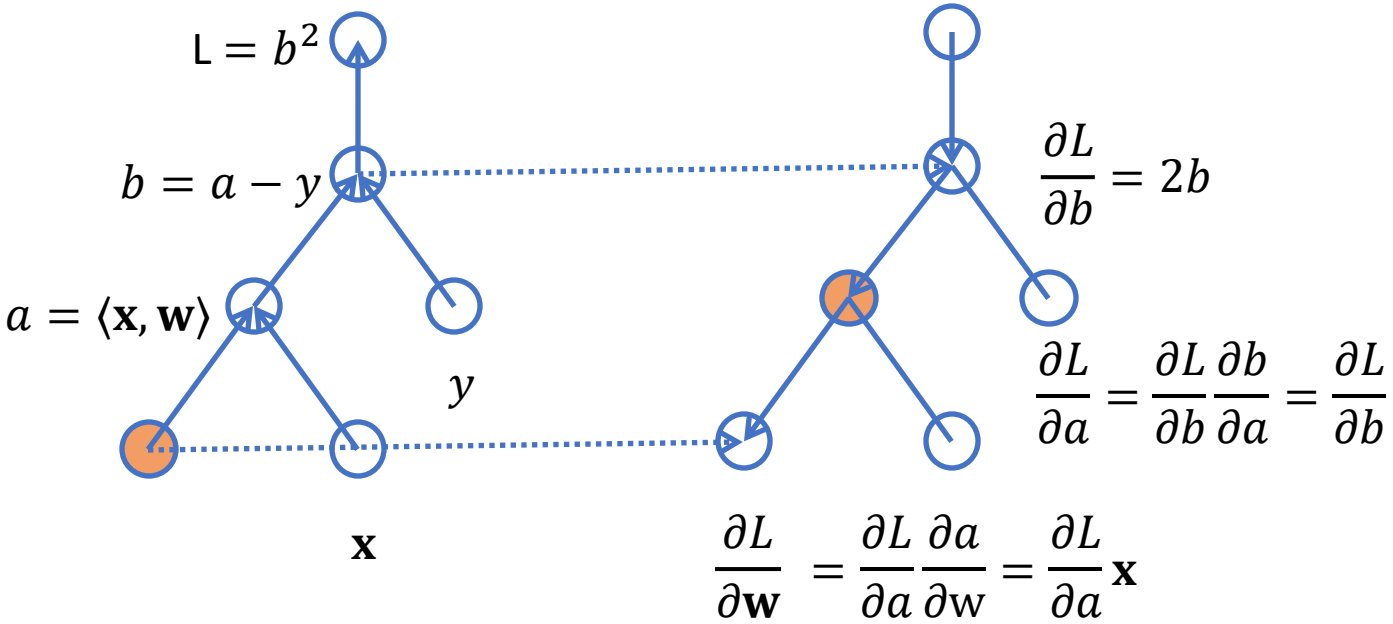
Reverse Accumulation

Assume $\mathrm{L} = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



Reverse Accumulation

Assume $\mathrm{L} = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



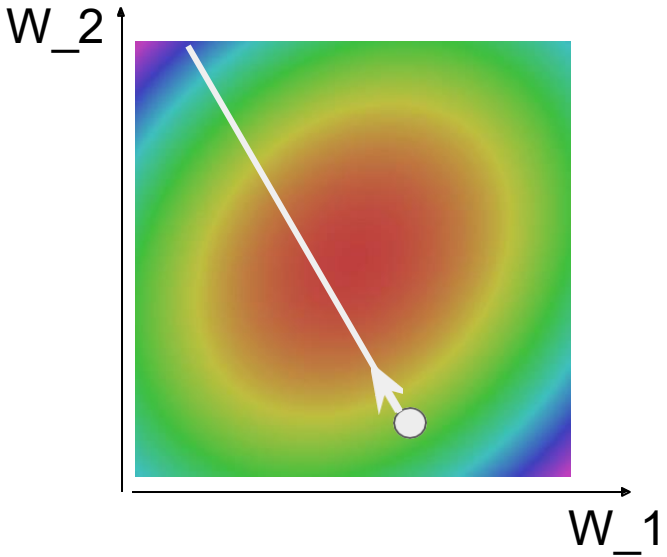
Optimizers

Newton’s Descent and Gradient descent

Gradient descent

```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```

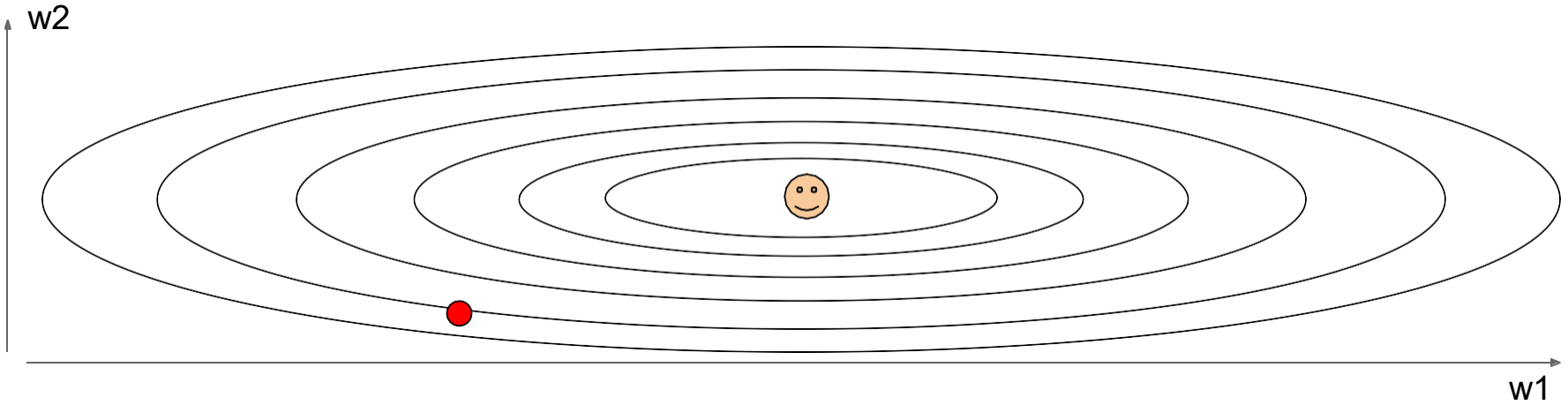


$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

J is the loss function here

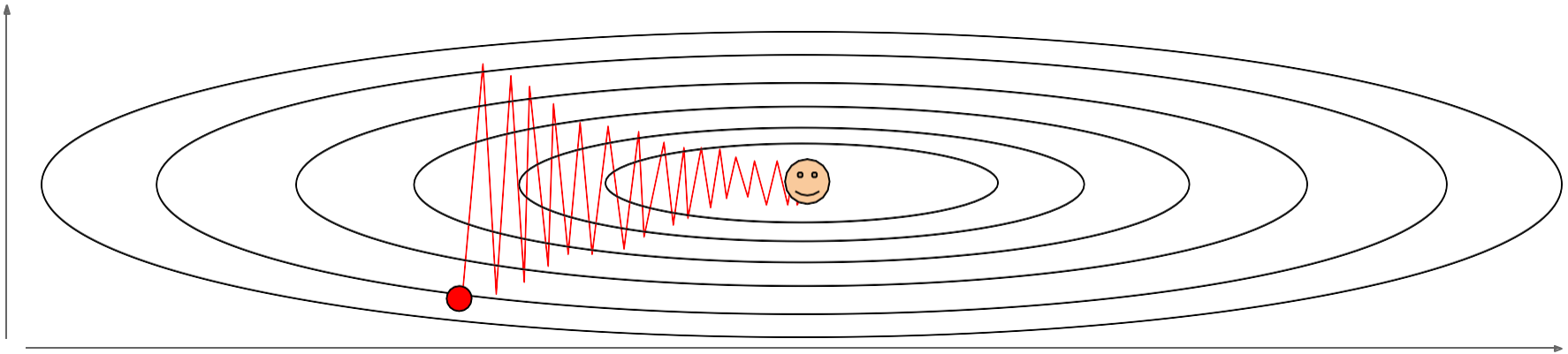
Optimization: Problem #1 with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?



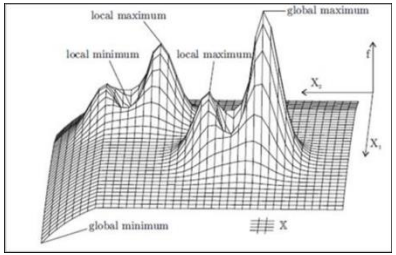
Optimization: Problem #1 with SGD

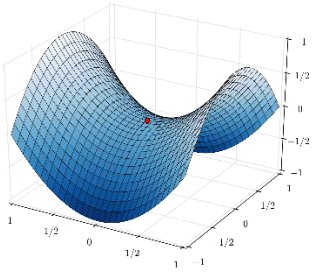
What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?
Very slow progress along shallow dimension, jitter along steep direction

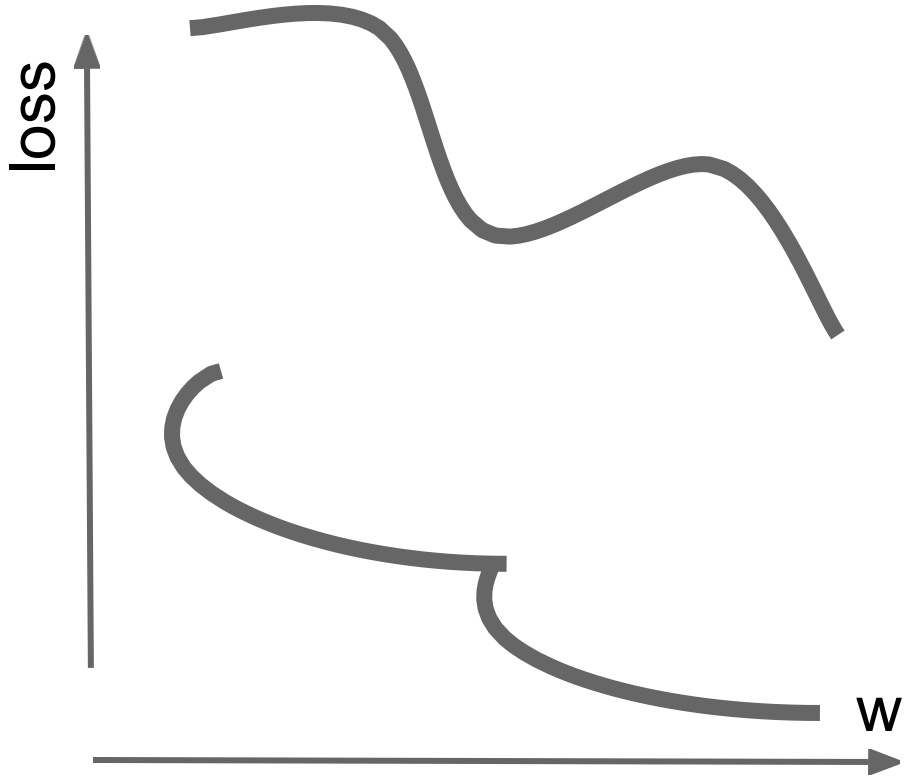


Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?



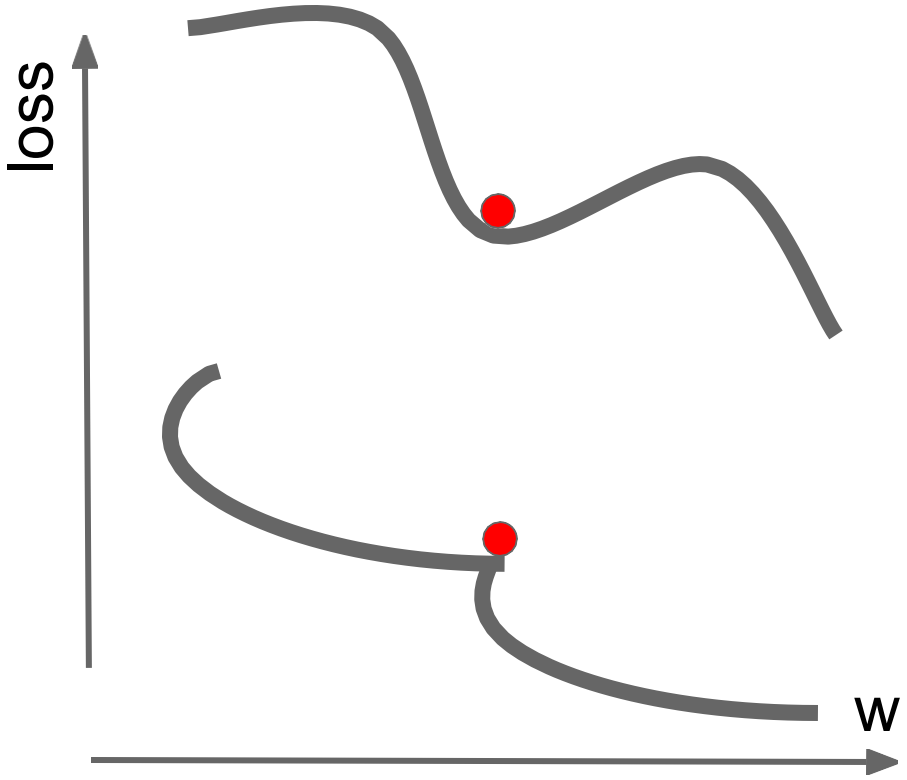




Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

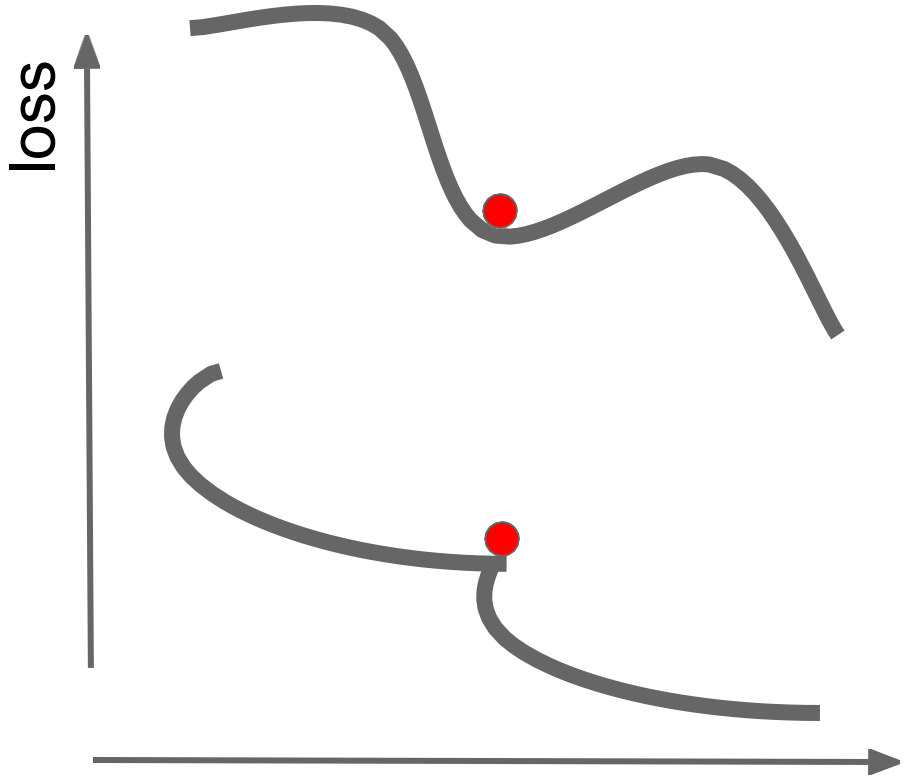
Zero gradient, gradient descent gets stuck



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

Saddle points much more common in high dimension



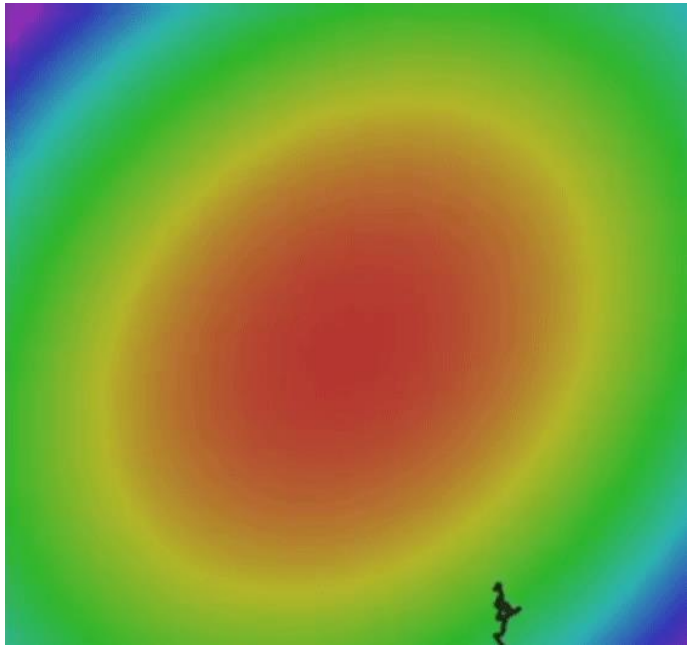
Dauphin et al, “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization”, NIPS 2014

Optimization: Problem #3 with SGD

Our gradients come from minibatches, so they can be noisy!

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W)$$



What are the Solutions?

1. Momentum (动量法) *(after-the-fact adjustment)*

1.1 Core Idea

Momentum accelerates gradient descent by **accumulating past gradients** to dampen oscillations in steep or noisy regions. It introduces a "velocity" term that smooths the optimization path.

1.2 Mathematical Formulation

- **Update Rule**:

$$\begin{cases} v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta J(\theta_t) \\ \theta_{t+1} = \theta_t - \alpha v_t \end{cases}$$

inertia 惯性

  - v_t: Velocity vector (weighted average of past gradients).
  - β: Momentum coefficient (typically **0.9**).
  - α: Learning rate.

Advantages	Disadvantages
Faster convergence in shallow regions.	May overshoot minima if β is too high.
Reduces oscillations in steep valleys.	Requires tuning β and α.

2. Nesterov Accelerated Gradient (NAG, 涅斯捷罗夫加速梯度) *(look-ahead adjustment)*

2.1 Core Idea

NAG improves Momentum by **"looking ahead"** before computing the gradient. Instead of evaluating the gradient at the current position (θ_t), it uses a **future position** ($\theta_t + \beta v_{t-1}$) to adjust the update direction.

2.2 Mathematical Formulation

- **Update Rule**:

$$\begin{cases} v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta J(\theta_t + \beta v_{t-1}) \\ \theta_{t+1} = \theta_t - \alpha v_t \end{cases}$$

  - The gradient is computed at $\theta_t + \beta v_{t-1}$ (future position).

Advantages	Disadvantages
More accurate gradient estimation.	Slightly more computation per step.
Reduces oscillations better than Momentum.	Still sensitive to hyperparameters.

Let’s see an example!
(Note example 5)

Adagrad

Adagrad **adapts learning rates per-parameter** based on historical gradient magnitudes. It assigns smaller updates to frequent features and larger updates to sparse features.

Mathematical Formulation

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \nabla_\theta J(\theta_t)$$

- G_t: Diagonal matrix of sum of squared past gradients ($G_t = G_{t-1} + (\nabla_\theta J(\theta_t))^2$).
- ϵ: Small constant (e.g., 1e-8) to avoid division by zero.

Key Properties

Advantages	Disadvantages
Works well for sparse data (e.g., NLP).	Learning rates may vanish over time.
No manual learning rate tuning needed.	Performs poorly for non-convex problems.

RMSProp

RMSProp fixes Adagrad's aggressive learning rate decay by replacing the sum of squared gradients with an **exponentially weighted moving average (EWMA)**.

Mathematical Formulation

$$\begin{cases} E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta)(\nabla_\theta J(\theta_t))^2 \\ \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} \odot \nabla_\theta J(\theta_t) \end{cases}$$

- $E[g^2]_t$: EWMA of squared gradients (β typically 0.9).

Key Properties

Advantages	Disadvantages
Solves Adagrad's vanishing LR issue.	Still requires tuning β and α.
Works well for non–stationary objectives (e.g., RNNs).	May need Momentum for some tasks.

Adam

It combines **Momentum (1st moment)** and **RMSProp (2nd moment)** for adaptive learning rates and smooth convergence.

Mathematical Formulation

$$\begin{cases} m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta J(\theta_t) \quad \text{(1st moment)} \\ v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta J(\theta_t))^2 \quad \text{(2nd moment)} \\ \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \quad \text{(bias correction)} \\ \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t}+\epsilon} \odot \hat{m}_t \end{cases}$$

- β_1: Momentum decay (default 0.9).
- β_2: Squared gradient decay (default 0.999).

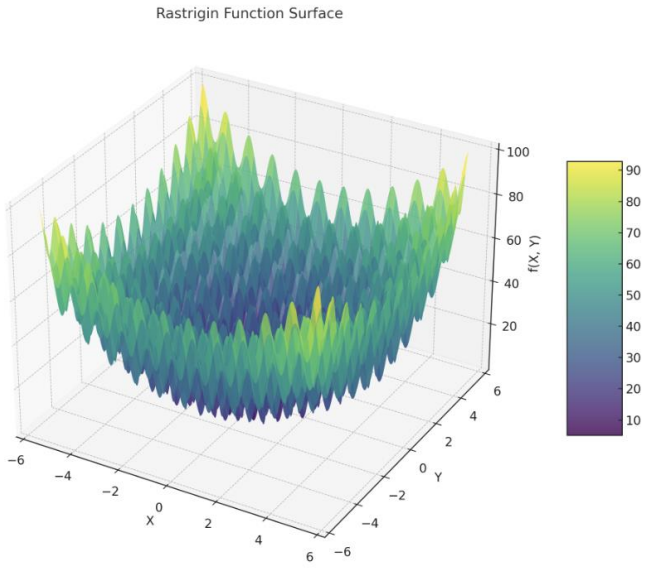
Key Properties

Advantages	Disadvantages
Fast convergence and stability.	More hyperparameters to tune (β_1, β_2, α).
Works well for deep learning (CNNs/Transformers).	May underperform if defaults are misused.

Optimizer	Mechanism	When to Use	Strengths	Weaknesses
Momentum	Gradient smoothing with velocity.	General convex optimization.	Reduces oscillations in valleys.	May overshoot minima.
NAG	"Looks ahead" before gradient step.	High-curvature landscapes.	Faster convergence than Momentum.	Slightly more computation per step.
Adagrad	Per-parameter adaptive LR (sum of squares).	Sparse data (e.g., NLP).	No manual LR tuning for sparse features.	LR vanishes over time.
RMSProp	EWMA of squared gradients.	Non-stationary tasks (e.g., RNNs).	Fixes Adagrad's LR decay.	Needs tuning β.
Adam	Momentum + RMSProp + bias correction.	**Deep learning (default choice).**	Fast and stable convergence.	Hyperparameter-sensitive.

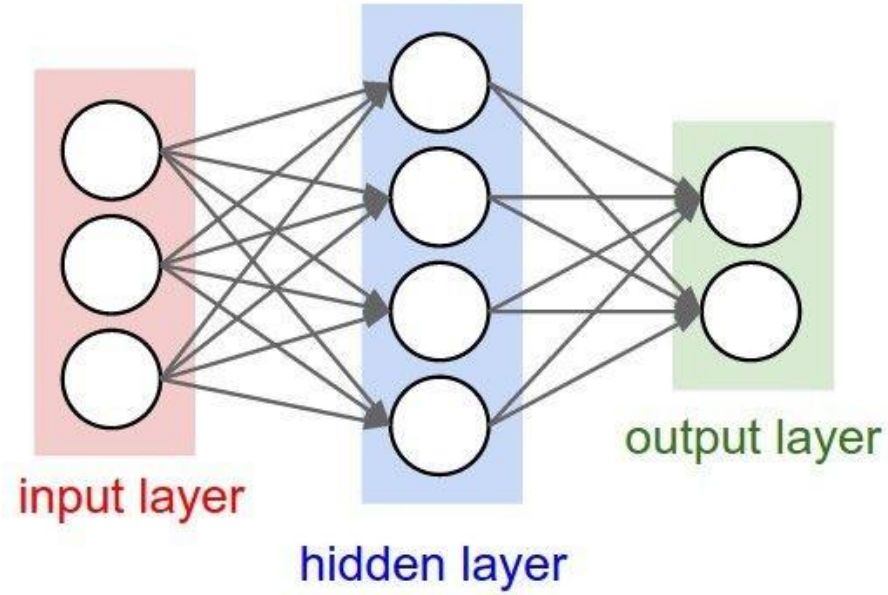
Let’s see an example!
(Note example 6)

$$f(x, y) = A \cdot 2 + \left[x^2 - A \cdot \cos(2\pi x)\right] + \left[y^2 - A \cdot \cos(2\pi y)\right]$$ A = 10



Weight Initialization

- Q: what happens when W=constant init is used?



All neurons could be **updated in the same way**, which prevents them to learn different things.

2. Random Initialization

Method

Sample weights from a uniform or normal distribution:

- **Uniform**: $W_{ij} \sim \mathcal{U}(-a, a)$
- **Normal**: $W_{ij} \sim \mathcal{N}(0, \sigma^2)$

Problems

- **Gradient Explosion/Vanishing**: Improper choice of a or σ can destabilize training.
- **Empirical Tuning Required**: Manual adjustment needed.

Improvement

- **Xavier/Glorot Initialization**: Scales variance based on layer dimensions (see below).

3. Xavier/Glorot Initialization

Principle

Proposed by Xavier Glorot, this method is designed for **saturating activation functions (e.g., Sigmoid, Tanh)**. It aims to maintain consistent variance of activations across layers.

Formulas

- **Uniform Distribution**:

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

- **Normal Distribution**:

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}\right)$$

Here, n_{in} and n_{out} are the input and output dimensions of the layer.

Advantages: 1) Stabilizes gradient variance during forward/backward passes;
2) Ideal for **Sigmoid** and **Tanh** activations; 3) Performs poorly with ReLU.

4. Kaiming Initialization

Introduced by Kaiming He, this method is optimized for **ReLU** and its variants (e.g., Leaky ReLU). It addresses the limitations of Xavier initialization for non–saturating activations.

Formulas

- **Normal Distribution**:

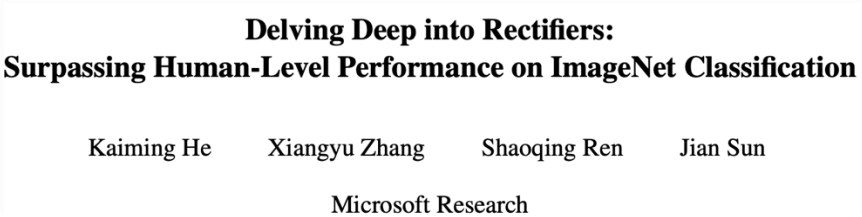
$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

- **Uniform Distribution**:

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right)$$

Advantages

- Resolves gradient vanishing in ReLU's negative regime.
- Suitable for deep networks (e.g., CNNs, ResNet).


Delving Deep into Rectifiers:
Surpassing Human-Level Performance on ImageNet Classification

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun

Microsoft Research

5. LeCun Initialization

Principle

Proposed by Yann LeCun for **SELU (Self-Normalizing Activation Functions)**.

Formula

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{1}{n_{\text{in}}}}\right)$$

Use Case

- SELU activations (self-normalizing networks).

$$\text{SELU}(x) = \lambda \cdot \begin{cases} x & \text{if } x > 0, \\ \alpha e^x - \alpha & \text{if } x \leq 0, \end{cases}$$

Let’s see an example!
(Note example 8)

Thank you!

Questions?